**Relatório 29 - Trabalho Final do Bootcamp: monte você mesmo (III)**

Matheus Beuren Carvalho

# 1. Introdução

O arroz é um alimento básico para cerca de 4,0 bilhões de pessoas (SILVA; WANDER, 2026) e as doenças que atingem as folhas como a queima bacteriana, a brusone, a mancha parda e o tungro, são um problema para quem produz o alimento. Nisso, o seguinte projeto trata da segmentação semântica de lesões em folhas de arroz, em que cada pixel da imagem recebe uma classe, o que mostra onde está a lesão e não só qual doença a folha tem como acontece na classificação de imagens feita pelo autor da base de dados(SETHY et al., 2020). Ao longo do relatório serão apresentados os conceitos usados como a arquitetura U-Net, os modelos pré-treinados, as funções de perda, métricas, o processamento dos dados, o treinamento e a comparação de nove modelos avaliados na base de validação e em uma base externa.

# 2. Desenvolvimento

A segmentação semântica atribui uma classe a cada pixel da imagem, esse tipo de segmentação foi escolhido porque o objetivo é identificar o tipo de lesão e observar a área afetada na folha. Diferente de outros tipos de segmentação como o de instância, que o objetivo é segmentar objetos, porém é tratado cada objeto como independente, ou seja, faz uma contagem de objetos de mesma classe diferente de considerar uma unica classe.

O encoder extrai as características da imagem, ele aplica convoluções que aumentam o número de filtros (feature maps) e a cada etapa reduz a imagem pela metade com ‘MaxPooling’, levando sempre para a próxima convolução apenas as características mais relevantes. O decoder pega essas características e gera a máscara de segmentação a partir das características extraídas no encoder, e em cada bloco a Conv2DTranspose aumenta a resolução na mesma escala em que o encoder diminuiu, fazendo assim a imagem voltar ao tamanho original.

As skips connections ligam os mapas de características do encoder aos do decoder de mesma resolução, pois o ‘MaxPooling’ podem perder detalhes e as skips conseguem recuperar alguns detalhes e dar ao decoder como referência para recriar a máscara. A saída da U-Net é uma função softmax por pixel com 5 canais, sendo quatro doenças e o fundo, onde cada pixel recebe uma probabilidade por classe e a classe escolhida é a de maior probabilidade.

O fine-tuning é a utilização de redes já treinadas de outras bases com uma taxa de aprendizado baixa para o modelo se adaptar aos dados.Neste trabalho o fine-tuning foi usado com encoders ResNet e EfficientNet treinados na ImageNet que tem cerca de 1,28 milhões de imagens em 1.000 classes diferentes e a vantagem é que eles já reconhecem bordas, texturas e cores de uma base grande.

A ResNet50 e a ResNet101 foram escolhidas por serem muito usadas em pesquisas de visão computacional, sendo o número de seu nome a profundidade da rede, ou seja, a quantidade de camadas que ela possui, e em redes da ResNet as conexões são residuais, onde em que cada bloco soma a sua entrada à sua saída e facilita o treino de redes profundas. A EfficientNetB0 é mais leve porque escala profundidade, largura e resolução em conjunto.

A Sparse Categorical Crossentropy é uma das métricas padrões ao se utilizar múltiplas saídas(softmax), ela atribui sua saída ao exato número da classe, não tendo a necessidade de transformar em on-hot como a CategoricalCrossEntropy. A loss Sparse penaliza o modelo quando ele dá uma probabilidade baixa à classe correta de cada pixel e o valor final é a média dessa penalidade sobre todos os pixels da imagem.

O Dice loss é baseado na métrica Dice em que medimos a sobreposição entre a máscara prevista e a real pela fórmula que multiplica duas vezes a interseção depois dividido pela soma das áreas das duas máscaras. A loss é 1 menos o Dice e como ela compara somente a região de cada classe, o fundo que ocupa a maior parte da imagem não domina o valor. A CE+Dice (Ma et al., 2021) é a soma das duas funções em que a crossentropy fornece um gradiente estável pixel a pixel e o dice foca na sobreposição da região(intercessão).

IoU (Intersection over Union) compara a máscara real com a prevista por meio da área da interseção dividido pela área da união das duas, com valor entre zero e um. O Dice usa duas vezes a interseção dividida pela soma das áreas e é sempre maior que o IoU, já que multiplica a intercessão.

$$IoU = \frac{Intercessão}{União}$$

$$Dice = \frac{2 * Intercessão}{União}$$

No projeto, as duas métricas são calculadas por classe e a média considera só as quatro doenças, pois o fundo é a classe predominante e se fosse calculado junto iria aumentar muito a média

A matriz de confusão compara a classe real (as linhas) com a prevista pelo modelo(colunas), e a diagonal mostra os acertos para a classe, e o restante mostra quais classes o modelo confunde entre si. A métrica ‘precision’ mede quanto do que o modelo previu como uma classe realmente era dela e o recall mede quanto da

classe o modelo encontrou. O F1 é a média harmônica das duas, ela ajuda a avaliar o modelo quando se possui classes minoritárias, ou quando não são tão reconhecidas no treino, e o F1 macro é a média simples entre as classes serve para avaliarmos como o modelo foi no geral de todas as classes. Suas respectivas fórmulas:

$$F1 = \frac{2 * precision * recall}{(precision + recall)}$$

$$F1\,macro = \frac{1}{C}\sum_{c=1}^{c} F1_C$$

Onde C é o número de classes.

A base de treino é a de Sethy et al. (2020), com 5.932 fotos de campo das quatro doenças coletadas em Odisha, e as máscaras são as da RiceSeg-5932, feitas manualmente e pareadas com as imagens pelo nome do arquivo. A base de teste externo é a de Hasan et al., da qual foram usadas só as quatro classes em comum com o treino, o que dá 2.623 imagens com um rótulo por imagem e sem máscara.

A classe de cada par vem da pasta da doença, numerada de zero a três (bacterial blight, blast, brown spot e tungro), e a classe quatro ficou para o fundo, de modo que cada amostra é uma tupla com o caminho da imagem, o caminho da máscara e o número da classe. As imagens foram lidas em RGB e redimensionadas para 288×288. A escolha destas dimensões se define por ao usar a arquitetura U-Net e RESNET para a segmentação da erro ao rodar o Concatenate, onde essa dimensionalidade não é múltiplo de 32.

As máscaras são arquivos ‘.jpg’ o que deixam os pixels ao redor da lesão com valores diferentes de 0 e 255. Por isso, cada máscara foi lida em escala de cinza, redimensionada com o vizinho mais próximo para não criar valores intermediários e binarizada com limiar 127(valor menor vira 0 e maior vira 1), ficando 1 onde há lesão e 0 no resto. No remapeamento de pixel para classe, o pixel de lesão recebe o número da doença da pasta e todo o resto recebe quatro(fundo). O resultado são arrays de imagens com formato (N, 288, 288, 3) e de máscaras com formato (N, 288, 288, 1) salvos em arquivos ‘.npy’, para que o processamento não precise ser refeito a cada treino e conseguir treinar os modelos mais fácil com os mesmos dados.

Os dados de treino(imagem e mascara) foram divididos de forma balanceada entre classes, em 80% para treino e 20% para validação, onde as duas partes tem a mesma proporção de cada doença.

A validação serve apenas para controlar o treino por meio de três callbacks, o ModelCheckpoint que salva o melhor modelo pelo IoU médio de validação, o EarlyStopping para o treino após oito épocas sem melhora na loss de validação e o ReduceLROnPlateau reduz a taxa de aprendizado após três épocas se nao tiver melhora. Como a validação vem da mesma base do treino ela tende a dar um resultado bom, e por isso a generalização é medida na base de teste externo.

Todos os modelos seguem a estrutura da arquitetura U-Net com entrada de 288×288×3 e saída de 288×288×5. O bloco de convolução é o mesmo em toda a rede e consiste em duas camadas Conv2D com kernel 3×3, cada uma seguida de BatchNormalization e ReLU, o decoder também é igual nos modelos sendo que cada bloco aplica uma Conv2DTranspose 2×2 que dobra a resolução, concatena o resultado com a skip da mesma resolução e passa pelo bloco de convolução, sendo a saída uma Conv2D 1×1 com softmax e cinco canais

Na U-Net sem pré-treino o encoder tem quatro estágios com 64, 128, 256 e 512 filtros(feature maps) cada um com o bloco de convolução e um MaxPooling que reduz a resolução pela metade, e a base tem 1.024 filtros. A imagem chega a 18×18 na base e o decoder devolve ao tamanho original com 512, 256, 128 e 64 filtros

Na ResNet50 e na ResNet101 o encoder é a rede pré-treinada na ImageNet sem a parte de classificação e com a entrada passando pelo ‘preprocess_input’ da própria ResNet, ele é utilizado para adequar a entrada das imagens da base de arroz com as que o encoder foi treinado. As skips connections e a base saem dos outputs das camadas de convolução da ResNet e a imagem de entrada é a skip de maior resolução. Em um dos testes, foi modificado a estrutura para que a ResNet101 ficasse com um decoder maior, porém usando o mesmo encoder e as mesmas skips com o decoder dobrado para 1.024, 512, 256 e 128 filtros, para testar se o decoder era o limite do desempenho. Na EfficientNetB0 o encoder também é pré-treinado na ImageNet e ele normaliza a imagem internamente, então não usa preprocess_input. Ele reduz a imagem cinco vezes e por isso o decoder tem cinco blocos com 512, 256, 128, 64 e 32 filtros.

Cada arquitetura foi treinada com duas funções de perda, a Sparse Categorical Crossentropy e a soma dela com o Dice loss. Como as máscaras são mapas de números inteiros de zero a quatro, a sparse recebe o rótulo direto sem converter para one-hot, e só o Dice precisa dessa conversão, que é feita dentro da própria loss. Os hiperparâmetros foram os mesmos para todos os modelos, com otimizador Adam, batch size de 16 e 50 épocas. A taxa de aprendizado inicial é 0,0001 na U-Net sem pré-treino e 0,00001 nos modelos com encoder pré-treinado, já que eles vêm com pesos da ImageNet. A métrica acompanhada durante o treino é o MeanIoU com cinco classes, não foi usado a acurácia por que um modelo que sempre prevê os pixels como fundo sempre vai ter acuracia alta.

Foram treinados nove modelos, que são as quatro arquiteturas com as duas funções de perda, e a ResNet101 com decoder maior treinada só com CE+Dice. Todos usam os mesmos dados, hiperparâmetros e callbacks e o que muda entre eles é o encoder, o decoder e a loss.

**Tabela 1 - IoU e Dice médio por classe**

| Modelo | Bacterialblight | Blast | Brownspot | Tungro | IoU | Dice |
|---|---|---|---|---|---|---|
| U-Net - SCE | 0,75 | 0,73 | 0,70 | 0,71 | 0,72 | 0,84 |
| U-Net - Dice e SCE | 0,75 | 0,74 | 0,71 | 0,72 | 0,73 | 0,84 |
| ResNet50 - SCE | 0,75 | 0,72 | 0,69 | 0,70 | 0,72 | 0,83 |
| ResNet50 - Dice e SCE | 0,75 | 0,72 | 0,69 | 0,71 | 0,72 | 0,83 |
| ResNet101 - SCE | 0,75 | 0,72 | 0,69 | 0,71 | 0,72 | 0,83 |
| ResNet101 - Dice e SCE | 0,75 | 0,72 | 0,69 | 0,70 | 0,72 | 0,83 |
| ResNet101 decoder maior - Dice e SCE | 0,75 | 0,72 | 0,69 | 0,71 | 0,72 | 0,83 |
| EfficientNetB0 - SCE | 0,74 | 0,72 | 0,68 | 0,69 | 0,71 | 0,83 |
| EfficientNetB0 - Dice e SCE | 0,74 | 0,71 | 0,69 | 0,70 | 0,71 | 0,83 |

A nomenclatura ‘SCE’ considera-se SparseCategoricalCrossEntropy.

O IoU médio ficou entre 0,71 e 0,73 e o Dice médio entre 0,83 e 0,84 uma diferença bem pequena entre os modelos, e o melhor foi a U-Net com Dice e SparseCategoricalCrossEntropy. Os encoders pré-treinados não superaram a U-Net sem pré-treino, e a EfficientNetB0 teve os menores valores. A Bacterialblight é a doença mais bem segmentada em todos os modelos e a Brownspot a pior na base de treino.

Na base externa só existe o rótulo da imagem e por isso a doença prevista para cada imagem foi a que teve mais pixels na máscara prevista, ou ‘sem lesão’ quando o modelo não previu nenhum pixel de lesão. A acurácia compara essa

doença com o rótulo da imagem e o F1, a precision e o recall são médias entre as quatro doenças.

**Tabela 2 - Métricas na base de teste.**

| Modelo | Acurácia | F1 macro | Precision macro | Recall macro |
|---|---|---|---|---|
| U-Net - SCE | 0,47 | 0,54 | 0,68 | 0,50 |
| U-Net - Dice e SCE | 0,43 | 0,46 | 0,62 | 0,48 |
| ResNet50 - SCE | 0,59 | 0,61 | 0,66 | 0,60 |
| ResNet50 - Dice e SCE | 0,58 | 0,60 | 0,66 | 0,60 |
| ResNet101 - SCE | 0,47 | 0,46 | 0,58 | 0,52 |
| ResNet101 - Dice e SCE | 0,54 | 0,56 | 0,63 | 0,57 |
| ResNet101 decoder maior - Dice e SCE | 0,55 | 0,57 | 0,64 | 0,57 |
| EfficientNetB0 - SCE | 0,51 | 0,50 | 0,63 | 0,53 |
| EfficientNetB0 - Dice e SCE | 0,51 | 0,55 | 0,66 | 0,54 |

O desempenho no teste externo ficou bem abaixo do desempenho esperado e variou muito entre os modelos, com acurácia de 0,43 a 0,59 e F1 macro de 0,46 a 0,61. O melhor modelo foi a ResNet50 com SparseCategoricalCrossEntropy e nenhum chegou à meta de 0,75 de acurácia. Como a base externa não tem máscaras, o Dice e o IoU não podem ser calculados nela e a meta de 0,70 só foi verificada na validação.

O efeito da função de perda não seguiu um padrão, a soma do Dice melhorou a ResNet101 com F1 de 0,46 para 0,56, e a EfficientNetB0,de 0,50 para 0,55, mas piorou a U-Net de 0,54 para 0,46 e mudou pouco a ResNet50 de 0,61 para 0,60. Como cada modelo foi treinado uma única vez, essas diferenças podem ser a diferença do treino e não efeito real da loss.

Entre os encoders a ResNet50 foi a mais eficiente nas duas funções de loss, com os maiores valores de acurácia e F1, enquanto a ResNet101 com

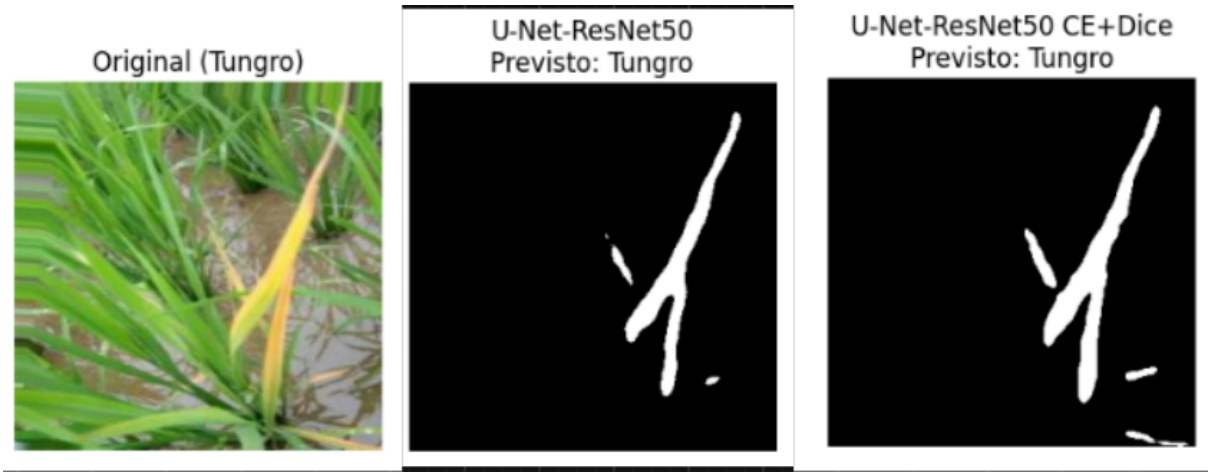
SparseCategoricalCrossEntropy teve o pior F1(possivelmente pela base pequena). Sendo assim, um encoder maior não garantiu melhor generalização, e na validação todos ficaram praticamente iguais.

Olhando a precision e o recall por doença, o padrão de erro foi parecido em todos os modelo, o tungro foi a doença mais bem reconhecida com recall entre 0,88 e 0,97 e o Blast teve a pior precision(0,27 a 0,40) o que mostra que muitas imagens de outras doenças foram classificadas como Blast. A Bacterialblight e a Brownspot tiveram recall baixo na maioria dos modelos, com valores de 0,11 a 0,73, ou seja, os modelos deixaram de reconhecer boa parte das imagens com essas classes.

A provável causa é que seja mudança de domínio(cenário) entre as duas bases, onde as imagens de treino possuem as lesões mais centralizadas e com foco nas lesões. Nos modelos pré-treinado houve um leve overfitting, a causa provável é talvez um grande modelo para a quantidade de dados que possui o treino.

Na figura 1 é mostrada imagens de teste e sua máscara prevista pelos dois melhores modelos

**Figura 1 - Imagem, mascara e previsão na base de teste**




## 3. Limitações e Trabalhos Futuros

Cada modelo foi treinado uma única vez, então pode ser que algumas diferenças entre eles podem ser ruído do treino e não efeito real da arquitetura ou da loss. O teste externo tem apenas o rótulo da imagem, sem máscara e por isso o Dice e o IoU só foram calculados na validação. Durante o treino foi possível ver o overfitting nos modelos com encoder pré-treinado, onde a metrica IoU de treino ficou bem acima da validação, como 0,85 contra 0,768 na ResNet101. Isso está ligado ao tamanho da base, que tem 4.745 pares de imagem e máscara no treino e é

pequena para encoders grandes, sendo que nenhum data augmentation foi aplicado e só foi testada a arquitetura U-Net, com três encoders.

Como trabalhos futuros fica aplicar data augmentation, repetir os treinos com parâmetros diferentes, testar arquiteturas de segmentação mais atuais e avaliar os modelos com uma base de imagens que possuem máscara, assim podendo medir o Dice e IoU.

# 4. Conclusão

O trabalho desenvolvido treinou e comparou nove modelos de segmentação semântica de lesões em folhas de arroz, combinando a U-Net com encoders ResNet50, ResNet101 e EfficientNetB0 e duas funções de loss. Na validação todos os modelos tiveram resultados consideravelmente bons. No teste externo o melhor modelo foi a ResNet50 com SparseCategoricalCrossEntropy, com acurácia de 0,59 e F1 macro de 0,61, abaixo da meta de 0,75. A soma do Dice à crossentropy não trouxe ganho real, e nem o decoder maior nem o encoder maior melhoraram a generalização. Com esses resultados, o projeto traz limitações encontradas durante o desenvolvimento e também ao realizar a avaliação dos modelos, nisso aplicações como o aumento da base de dados de treino e o uso de uma base com máscara no teste, seriam as possíveis soluções para alcançar melhoras no resultado.